1 Introduction

This report implements and analyses a binary classification pipeline using PyTorch applied to the Breast Cancer Wisconsin dataset [1]. A feedforward neural network is trained and evaluated against a logistic regression baseline, and a series of experiments investigate model performance. A fixed random seed of 42 was applied across all experiments to ensure full reproducibility.

2 Dataset Description

2.1 Overview

The Breast Cancer Wisconsin dataset was accessed via `scikit-learn`. The dataset comprises 569 samples, each described by 30 numerical features. Each sample carries a single binary label. The classification task is to predict whether a tumour is *malignant* (class 0) or *benign* (class 1).

2.2 Class Distribution

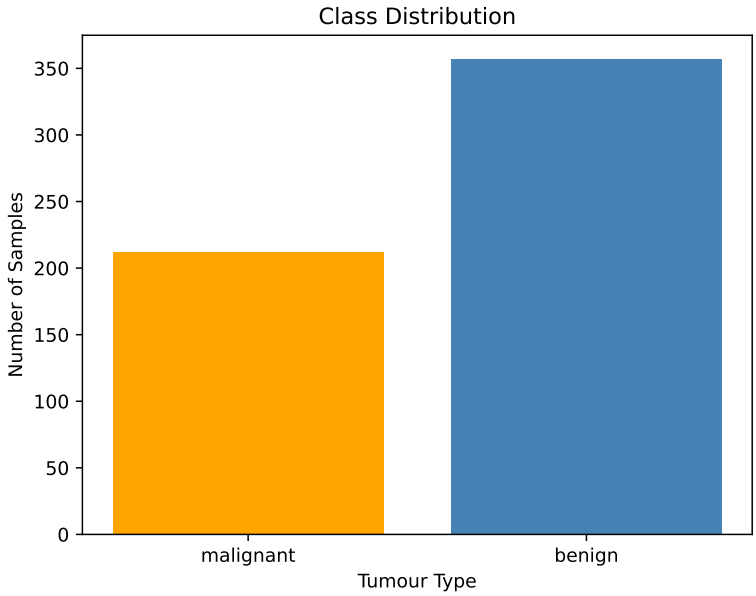


Figure 1: Class distribution of the Breast Cancer Wisconsin dataset.

Of the 569 samples, 212 (37%) are labelled malignant and 357 (63%) benign. The dataset is mildly imbalanced. However, the imbalance ratio is 1:1.68 and can be treated as negligible such that we can continue with a standard modelling approach.

3 Train/Test Split and Data Preprocessing

3.1 Train/Test Split

Machine learning models must be evaluated on data that was not used during training to obtain an unbiased estimate of generalisation performance. Exposing the model to test data during training would lead to overoptimistic performance estimates that do not reflect real-world deployment.

An 80/20 split was chosen, yielding 455 training samples and 114 test samples. A 70/30 split would waste learning data on a dataset of this size, while a 90/10 split would provide too few test samples (57) to obtain stable performance estimates. The `stratify=y` argument was passed to `train_test_split` to ensure a balance of classes when sampling

3.2 Feature Scaling

Neural networks optimise via gradient descent, which is sensitive to the scale of input features. Large numerical features dominate, causing training to become slow and unstable. `StandardScaler` was applied to standardise each feature to zero mean and unit variance.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # fit only on training data
X_test_scaled  = scaler.transform(X_test)       # apply same transform to test
```

Critically, the scaler is fitted on the training set only and applied to the test set using the same parameters, preventing data leakage from the test distribution into the training process. The scaled arrays were then converted to PyTorch float tensors via `torch.tensor(..., dtype=torch.float32)` to interface with the PyTorch training pipeline.

4 Neural Network Architecture

4.1 Architecture Design

A feedforward neural network was implemented in PyTorch using the following architecture:

Layer	Type	Neurons	Activation	Parameters
Input	Linear	$30 \to 64$	ReLU	$(30 \times 64) + 64 = 1{,}984$
Hidden	Linear	$64 \to 32$	ReLU	$(64 \times 32) + 32 = 2{,}080$
Output	Linear	$32 \to 1$	Sigmoid	$(32 \times 1) + 1 = 33$
Total				**4,097**

Table 1: Network architecture and parameter counts.

4.2 Justification of Design Choices

Number of hidden layers. two hidden layers allows the network to model feature interactions more efficiently — the first layer captures patterns in raw features, the second layer combines these into higher-order representations. **Neuron counts.** The first hidden layer is wide enough to capture the diversity of input features . The second layer acts as a compression step, retaining only the most important feature interactions and discarding noise. **ReLU activation.** The Rectified Linear Unit, $\text{ReLU}(x) = \max(0, x)$, introduces nonlinearity, allowing the network to learn nonlinear decision boundaries. ReLU avoids the vanishing gradient problem, since its gradient is 1 for all positive inputs. **Sigmoid output.** The output layer applies a sigmoid activation to squash the raw output to the range $(0, 1)$, producing a probability estimate. This directly matches the Bernoulli probabilistic assumption $Y_i \sim \text{Bernoulli}(p_i)$ underlying the binary cross-entropy loss derived in Section 5. **Parameter count.** The total 4097 trainable parameters is conservative relative to the 455 training samples, reducing the risk of memorisation

5 Loss Function and Optimiser

5.1 Probabilistic Model and Likelihood

Each label $Y_i \in \{0, 1\}$ is modelled as a Bernoulli random variable with success probability $p_i = f_\theta(X_i)$, the output of the network after sigmoid activation:

$$Y_i \sim \text{Bernoulli}(p_i), \quad p_i = f_\theta(X_i) \in (0, 1). \tag{1}$$

The probability of observing a single label y_i is:

$$P(Y_i = y_i) = p_i^{y_i}(1 - p_i)^{1-y_i}. \tag{2}$$

Assuming independent observations, the *likelihood* of observing all labels simultaneously is:

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i}. \tag{3}$$

5.2 Log-Likelihood and Binary Cross Entropy Equivalence

Maximising $\mathcal{L}(\theta)$ directly causes numerical underflow; since log is monotonically increasing, we equivalently maximise the *log-likelihood*:

$$\ell(\theta) = \sum_{i=1}^{n} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right]. \tag{4}$$

Gradient-based optimisers minimise rather than maximise. Negating and normalising by n:

$$-\frac{\ell(\theta)}{n} = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right] =: \mathcal{L}(\theta). \tag{5}$$

Since negation and scaling by $\frac{1}{n}$ are monotone transformations, the minimiser θ^* of $\mathcal{L}(\theta)$ is identical to the maximiser of $\ell(\theta)$.

5.3 What the Loss Measures

BCE measures the discrepancy between predicted probabilities and true labels. The logarithmic form penalises confident incorrect predictions more heavily than uncertain ones. This encourages well-calibrated estimates

5.4 Why an Optimiser is Required

There is no closed-form solution for neural network weights — the loss surface is high-dimensional and non-convex. An optimiser iteratively computes $\nabla_\theta \mathcal{L}$ via backpropagation and updates parameters in the direction of steepest descent: $\theta \leftarrow \theta - \eta \cdot \nabla_\theta \mathcal{L}$.

5.5 Optimiser: Adam

The Adam optimiser (Adaptive Moment Estimation) was selected [2]. Adam was preferred over SGD because: (i) it adapts the learning rate individually per parameter; (ii) it converges faster on small datasets such as this one where gradient estimates can be noisy.

5.6 Role of the Learning Rate

The learning rate $\eta = 0.001$ controls the step size at each gradient descent iteration. If η is too large, updates may miss good solutions, too samall and convergence is slow. The value 0.001 is the well-established default for Adam.

```
model     = Net(input_dim=X_train.shape[1])
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

6 Model Training

6.1 Training Procedure

The model was trained for 50 epochs using full-batch gradient descent. Each epoch consists of a forward pass, loss computation, backpropagation, and parameter update.

6.2 Training Performance

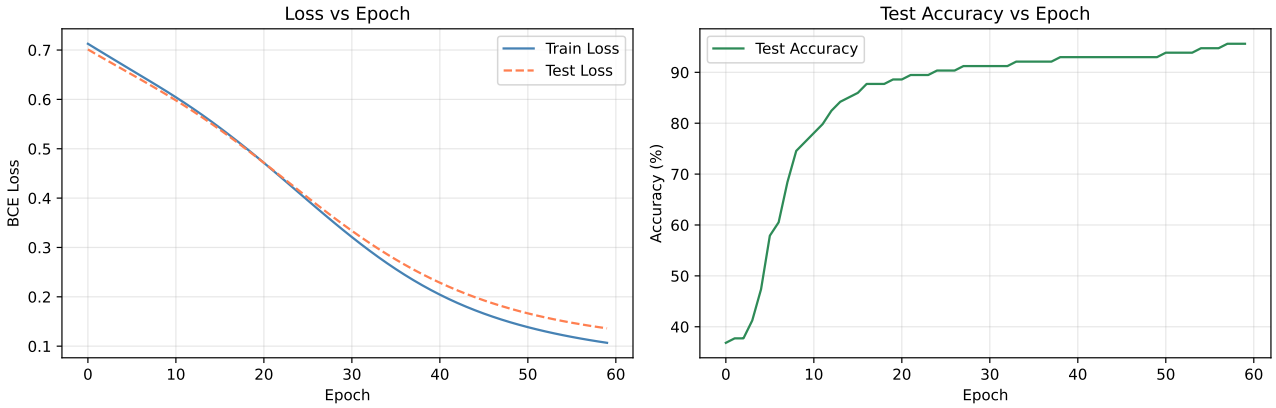


Figure 2: Training and test loss (left) and test accuracy (right) over 50 epochs.

Both training and test loss decreased consistently across all 50 epochs, confirming successful learning. At epoch 0 the training and test losses are 0.7127 and 0.7013, respectively. By epoch 50 the training loss had fallen to 0.1385 and test loss to 0.1665. The rate of decrease was fastest in early epochs, slowing as the model approached convergence — characteristic of gradient descent as gradients shrink near a minimum.

The test loss tracked the training loss closely throughout, with a gap of 0.030 at epoch 50. This small gap indicates **no meaningful overfitting** at this training duration — the model generalises well to unseen data. The test loss is consistently slightly higher than the training loss, indicating the model performs better on previously seen data.

7 Model Evaluation

7.1 Performance Metrics

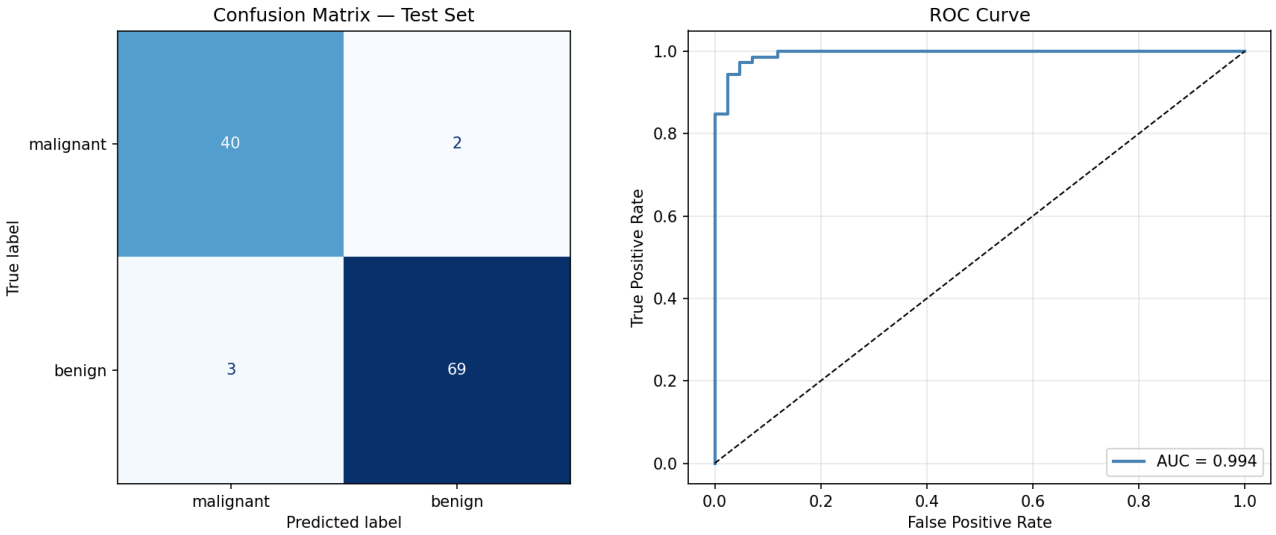


Figure 3: Confusion matrix (left) and ROC curve (right) for the baseline neural network.

Model	Accuracy	Precision	Recall	F1
Neural Network	95.61%	0.9718	0.9583	0.9650

The model correctly classified 109 of 114 test samples. A recall of 0.9583 on the malignant class means the model correctly identified 95.83% of true malignant cases, missing 2 out of 42. In a clinical

screening context this is an important metric, as false negatives carry greater consequences than false positives. The model demonstrates strong generalisation, achieving test performance consistent with its training behaviour.

8 Baseline: Logistic Regression

Logistic regression is a linear classifier. It computes a weighted sum of input features:

$$z = w^\top x + b, \tag{6}$$

and applies a sigmoid function to produce a class probability $p = \sigma(z)$. Its decision boundary is a single hyperplane in feature space; it cannot capture nonlinear class boundaries.

8.1 Results and Comparison

Model	Accuracy	Precision	Recall	F1
Logistic Regression	98.25%	0.9861	0.9861	0.9861
Neural Network	95.61%	0.9718	0.9583	0.9650
Difference	−2.63%			

The logistic regression baseline outperformed the neural network by 2.63%, correctly classifying 112 of 114 test samples. The neural network therefore does **not** provide a meaningful improvement over the simpler model on this dataset.

This outcome is likely explained by the nature of the data. The Breast Cancer dataset consists of 30 clinical features that are structured and largely linearly separable. A single linear decision boundary is sufficient to classify the majority of samples correctly, leaving no advantage for the neural network's nonlinear capacity. Furthermore, logistic regression benefits from convex optimisation with a guaranteed global minimum, whereas gradient descent on the neural network's non-convex loss surface is not guaranteed to find an equally good solution. The 5 samples consistently misclassified by the neural network appear to be borderline cases whose feature values sit close to the decision boundary; the linear boundary of logistic regression happens to align more favourably with these cases on this particular split.

9 Experiments

9.1 Experiment 1: Extended Training (200 Epochs)

Motivation. The 50-epoch model's loss was still decreasing at the final epoch. Extending training to 200 epochs investigated whether the neural network was underfitting and could close the gap with logistic regression through longer optimisation.

Results.

Epoch	Train Loss	Test Loss	Gap	Test Accuracy
50	0.1385	0.1665	0.028	95.61%
190	0.0292	0.0860	0.058	95.61%

Discussion. Despite the training loss falling to 0.0292, the test accuracy remained at 95.61% — identical to the 50-epoch model. The train/test loss gap more than doubled from 0.028 to 0.058, confirming **overfitting**: the model continued to minimise training loss by fitting noise and dataset-specific patterns that do not generalise. The accuracy plateau confirms that the 5 misclassified samples represent a hard ceiling determined by the data rather than training duration. This result rules out underfitting as an explanation for the neural network's underperformance relative to logistic regression.

9.2 Experiment 2: Dropout Regularisation

Motivation. Since the 200-epoch experiment confirmed mild overfitting, dropout regularisation was applied to reduce the train/test gap. Three dropout rates were evaluated: 0.1, 0.2, and 0.3.

Results.

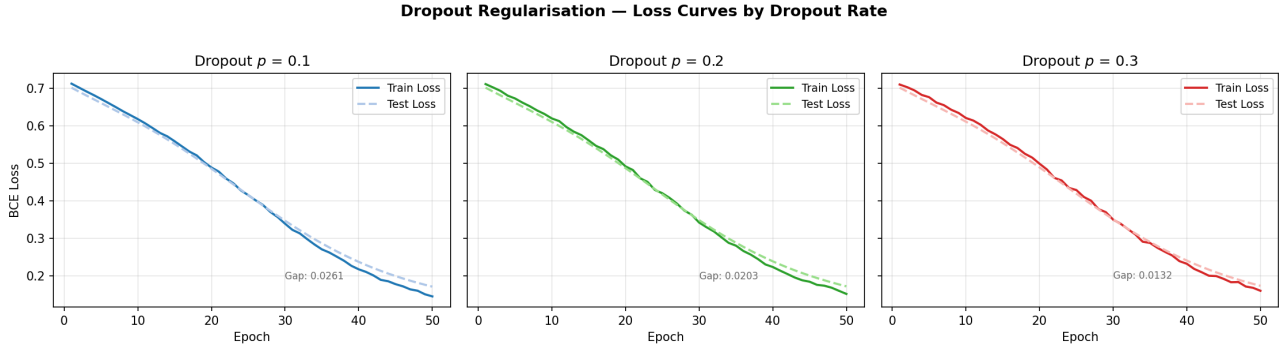


Figure 4: Training and test loss for dropout rates 0.1, 0.2, and 0.3.

Model	Train Loss	Test Loss	Gap	Accuracy	F1
Original NN	0.1385	0.1665	0.028	95.61%	0.9650
Dropout $p = 0.1$	0.1443	0.1659	0.022	95.61%	0.9650
Dropout $p = 0.2$	0.1528	0.1666	0.014	94.74%	0.9583
Dropout $p = 0.3$	0.1623	0.1678	0.006	94.74%	0.9583

Discussion. As dropout rate increases, the train/test loss gap decreases monotonically from 0.028 to 0.006, confirming that dropout is functioning as intended. However, rates of 0.2 and 0.3 reduce test accuracy by 0.87%, corresponding to one additional misclassification. This is the classic **bias-variance tradeoff**: higher dropout reduces variance (overfitting) but increases bias (underfitting) by reducing the effective capacity of the already-compact network. At dropout $p = 0.1$, the model achieves the same accuracy as the original while reducing the train/test gap from 0.028 to 0.022, making it the best-performing configuration. This demonstrates that the original network had only very mild overfitting, and aggressive regularisation is counterproductive on a dataset of this complexity.

10 Discussion and Conclusion

Across all experiments the 95.61% accuracy ceiling proved to be robust with every iteration of the model failing to correctly classify these borderline cases, suggesting the limit is data - driven rather than model - driven.

10.1 Limitations

The dataset is structured, low-dimensional, and largely linearly separable — conditions that favour logistic regression over neural networks. The single 80/20 train/test split means the observed 2.63% accuracy gap corresponds to approximately 3 samples.

10.2 Improvements

Cross-validation. Replacing the single split with k-fold cross-validation ($k = 5$ or $k = 10$) would produce statistically robust performance estimates and reduce sensitivity to the particular split chosen.

Feature selection. Principal component analysis or feature importance analysis could identify and remove redundant features among the 30 inputs, potentially simplifying the decision boundary and closing the gap with logistic regression.

References

[1] W. N. Street, W. H. Wolberg, O. L. Mangasarian, *Nuclear feature extraction for breast tumor diagnosis*, Proc. SPIE 1993.

[2] D. P. Kingma, J. Ba, *Adam: A Method for Stochastic Optimization*, ICLR 2015.

[3] K. Hornik, M. Stinchcombe, H. White, *Multilayer feedforward networks are universal approximators*, Neural Networks, 1989.